

Bitácora de Práctica: Arquitectura de Microservicios (Mexi-Banco)

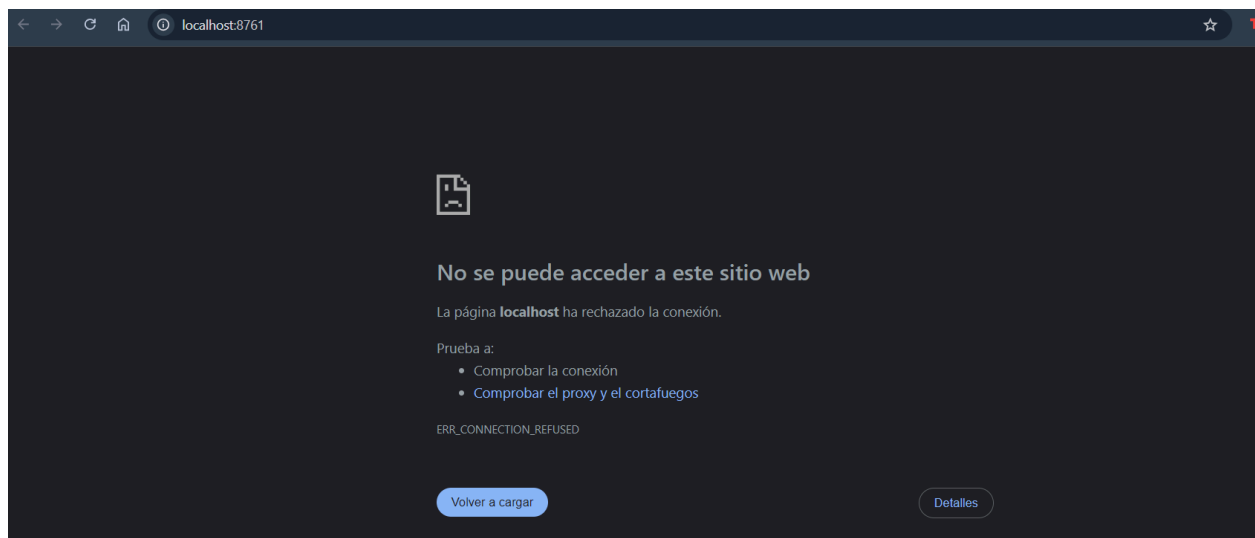
1. La historia (15:30 hrs)

Qué se rompió al pasar a HTTP: Al separar el monolito en microservicios, se rompió la transacción única (@Transactional) de la base de datos. Antes, si una operación fallaba, todo se deshacía automáticamente porque compartían memoria y base de datos. Ahora, mediante HTTP, cada servicio confirma solo su parte; si la transferencia falla después de hacer el cargo, este ya no se deshace automáticamente, dejando los datos inconsistentes.

2. Implementación del Service Discovery (Eureka) (15:50 hrs)

El objetivo de este paso fue eliminar la dependencia de direcciones IP estáticas. Para ello, habilitamos el servidor Eureka con la anotación @EnableEurekaServer y configuramos los 5 microservicios para que se registraran automáticamente en él.

- **El tablero antes (no carga):** Antes de encender el contenedor discovery, el directorio no existía. Al intentar entrar a localhost:8761, el navegador arrojaba un error de conexión.



Captura 1: No se puede acceder a este sitio web

- **El tablero después (cinco servicios UP):** Una vez levantados los contenedores, verificamos el registro exitoso. En el tablero de Eureka ya se observan los 5 servicios (CUENTA, MOVIMIENTO, NOTIFICACION, SPEI, TRANSFERENCIA) listados en estado UP (1).

The screenshot shows the Spring Eureka dashboard at localhost:8761. The 'System Status' section indicates that the system is healthy, with a current time of 2018-10-26T15:55:07+0000. The 'DS Replicas' section shows 10 replicas. The 'Instances currently registered with Eureka' section lists five instances: CUENTA, MOVIMIENTO, NOTIFICACION, SPD, and TRANSFERENCIA, all with a status of UP. The 'General Info' section shows system metrics like total available memory (8192) and registered instances (5). The 'Instance Info' section shows details for the CUENTA instance, including its IP (172.20.0.8) and status (UP).

Application	Hosts	Availability Zones	Status
CUENTA	172.20.0.8	(1)	UP (1) - 172.20.0.8:8080
MOVIMIENTO	172.20.0.8	(1)	UP (1) - 172.20.0.8:8080
NOTIFICACION	172.20.0.8	(1)	UP (1) - 172.20.0.8:8080
SPD	172.20.0.8	(1)	UP (1) - 172.20.0.8:8080
TRANSFERENCIA	172.20.0.8	(1)	UP (1) - 172.20.0.8:8080

Captura 2: Primeros 5 microservicios registrados en estado UP

- **El XML de CUENTA:** Al consultar directamente la API del directorio, comprobamos que Eureka guarda la información del microservicio Cuenta en formato XML, exponiendo correctamente su IP interna, puerto (8080) y su estado UP.

This XML file does not appear to have any style information associated with it. The document

```

<?xml version="1.0" encoding="UTF-8"?>
<application>
  <name>CUENTA</name>
  <instance>
    <instanceId>c584711edef1:cuenta:8080</instanceId>
    <hostname>172.20.0.8</hostname>
    <app>CUENTA</app>
    <ipAddr>172.20.0.8</ipAddr>
    <status>UP</status>
    <overriddenstatus>UNKNOWN</overriddenstatus>
    <port enabled="true">8080</port>
    <securePort enabled="false">443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
      <name>MyOwn</name>
    </dataCenterInfo>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791561207314</registrationTimestamp>
      <lastRenewalTimestamp>1791561661350</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791561206624</serviceUpTimestamp>
    </leaseInfo>
    <metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.20.0.8:8080</homePageUrl>
    <statusPageUrl>http://172.20.0.8:8080/actuator/info</statusPageUrl>
    <healthCheckUrl>http://172.20.0.8:8080/actuator/health</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791561207314</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791561206033</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
</application>

```

Captura 3: Estructura XML con el status UP

- **Respuesta a la pregunta "Piensa":** ¿Alguien está usando el directorio para encontrar a cuenta? No. Aunque los servicios ya se anotaron en el directorio, hasta este paso de

la práctica nadie lo está usando para buscar direcciones. El servicio de transferencia todavía tiene la dirección escrita a mano (`http://cuenta:8080`) en el archivo `docker-compose.yml`, por lo que sigue usando la red estática de Docker en lugar de preguntarle a Eureka.

3. Implementación del API Gateway (16:15 hrs)

- **Objetivo:** Proveer un único punto de entrada para los clientes externos (puerto 8080), encargándose de enrutar las peticiones al microservicio correspondiente.
- **Configuración:** Se corrigieron errores de sintaxis y comentarios en el archivo `docker-compose.yml` para habilitar el contenedor del gateway. Se definieron las reglas de enrutamiento (Predicates) en el archivo `application.yml` del Gateway, estableciendo que las rutas como `Path=/cuentas/**` utilicen balanceo de carga consultando a Eureka (`uri: lb://CUENTA`).

```
8 gateway:
9   # Cuanto espera el gateway a un servicio. Sin esto, un servicio
10  httpclient:
11    connect-timeout: 2000
12    response-timeout: 15s
13    # Una ruta = predicados (cuando aplica) + uri (a donde va) + fi
14    # lb://cuenta significa: "preguntale a Eureka por las instancia
15    # ellas" (round robin). StripPrefix=1 quita /api antes de reenv
16    # cuenta como /cuentas/1, que es lo que cuenta sabe atender.
17    #
18    # Solo se publican los endpoints para el cliente. Los internos
19    # y /abonos, POST /movimientos, POST /notificaciones) no tienen
20    # Por eso cuenta usa {clabe} (un solo segmento) y no /**, y mov
21    # aceptan GET. Siguen abiertos dentro de la red de Compose, que
22  routes:
23    - id: cuenta
24      uri: lb://cuenta
25      predicates:
26        - Path=/api/cuentas,/api/cuentas/{clabe}
27      filters:
28        - StripPrefix=1
29    - id: movimiento
30      uri: lb://movimiento
31      predicates:
32        - Path=/api/movimientos
33        - Method=GET
34      filters:
35        - StripPrefix=1
36    - id: transferencia
37      uri: lb://transferencia
38      predicates:
39        - Path=/api/transferencias,/api/transferencias/{id}
40      filters:
41
42    - id: notificacion
43      uri: lb://notificacion
44      predicates:
45        - Path=/api/notificaciones
46        - Method=GET
47      filters:
48        - StripPrefix=1
49    - id: spei
50      uri: lb://spei
51      predicates:
52        - Path=/api/spei
53        - Method=POST
54      filters:
55        - StripPrefix=1
```

Captura 4: Gateway con las rutas correctamente añadidas

- **Verificación:** Se comprobó que, en lugar de consultar directamente al servicio de cuentas en el puerto 8081, ahora es posible realizar la consulta a través de la puerta principal `localhost:8080/cuentas/000`, obteniendo la respuesta correcta desde el microservicio aislado en el backend.
- **El error 400 del 8082 (Acceso directo sin gateway)** Para comprobar que los microservicios seguían expuestos antes de aislar la red, intentamos hacer un registro de movimiento apuntando directamente al puerto del servicio (8082). Como se observa a continuación, el servicio responde, arrojando un error 400. Esto demuestra que, sin el gateway, cualquiera podría acceder a operaciones que deberían ser estrictamente

internas.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Sat, 10 Oct 2026 04:52:53 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
```

Captura 5: Terminal con error 400

Análisis de los dos errores 404 por el Gateway:

- **El 404 producido por CUENTA:** Al consultar localhost:8080/api/cuentas/000, el 404 lo devuelve el microservicio Cuenta. Sabemos esto porque la respuesta JSON incluye la clave "instance": "/cuentas/000" (o cabecera X-Instancia), demostrando que el gateway enrutó bien la petición pero la base de datos no encontró el registro.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: e2c9fa37c675
Content-Type: application/problem+json
Date: Sat, 10 Oct 2026 05:09:06 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
```

Captura 6: Consulta a localhost:8080/cuentas/000

- **El 404 producido por el GATEWAY:** Al intentar un POST a localhost:8080/api/movimientos, el 404 lo arroja directamente el Gateway. Sabemos esto porque no hay rastro de la instancia del microservicio; el gateway bloquea la petición al no existir una ruta predicates que permita el método POST hacía movimientos.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 132

{"timestamp":"2026-10-10T05:02:46.471+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"4bf841f6-1478"}
```

Captura 7: Terminal con error 404

Alta (201) y Consulta (200) exitosa Finalmente, para comprobar que las rutas permitidas funcionan correctamente, creamos una cuenta de prueba generando una variable LAB_CLABE con la hora del sistema. La petición pasó por el gateway exitosamente devolviendo un 201

Created, y al consultarla inmediatamente después, obtuvimos un 200 OK con el saldo en cero.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ LAB_CLABE="S07$(date +%s)"
curl -i -X POST http://localhost:8080/api/cuentas \
  -H 'Content-Type: application/json' \
  -d '{"clabe":"$LAB_CLABE","titular":"S07","saldoInicial":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 74a7fcbb3464
Content-Type: application/json
Date: Sat, 10 Oct 2026 05:11:30 GMT

{"id":3,"clabe":"S071791609089","titular":"S07","saldo":0}
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ curl -i "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 74a7fcbb3464
Content-Type: application/json
Date: Sat, 10 Oct 2026 05:14:28 GMT

{"id":3,"clabe":"S071791609089","titular":"S07","saldo":0.00}
```

Captura 8: Alta y Consulta

4. Replicación y Seguridad de Red Interna (16:30 hrs)

- **Eliminación de puertos expuestos:** Con el Gateway funcionando como puerta principal, se eliminó la exposición directa de puertos (ports: - "808X:8080") en los servicios del backend. Ahora estos servicios son inaccesibles desde el exterior de la red de Docker.
- **Escalabilidad horizontal:** Se modificó la configuración del servicio cuenta para desplegar 3 instancias simultáneas (deploy: replicas: 3). La eliminación del mapeo de puertos evitó el choque (colisión) entre estas tres instancias, permitiendo que cada una escuche internamente en su propio puerto 8080.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS                               PORTS
mexi-banco-v06-cuenta-1            mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     2 minutes ago  Up 2 minutes (healthy)             8080/tcp
mexi-banco-v06-cuenta-2            mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     2 minutes ago  Up 2 minutes (healthy)             8080/tcp
mexi-banco-v06-cuenta-3            mexi-banco-v06-cuenta              "java -jar app.jar"                  cuenta     2 minutes ago  Up 2 minutes (healthy)             8080/tcp
mexi-banco-v06-db-1                 postgres:17-alpine                 "docker-entrypoint.sh"               db         44 hours ago   Up 44 hours (healthy)              5432/tcp
mexi-banco-v06-discovery-1         sha256:71d2fa099b62e61b1c3c84a49862dd527cedef585acaba4d39a2f69b1db8807 "java -jar app.jar"                  discovery  17 hours ago   Up 17 hours (healthy)              0.0.0.0:8761->8761/tcp
mexi-banco-v06-gateway-1           mexi-banco-v06-gateway             "java -jar app.jar"                  gateway    36 minutes ago  Up 36 minutes (healthy)            0.0.0.0:8080->8080/tcp
mexi-banco-v06-movimiento-1        mexi-banco-v06-movimiento          "java -jar app.jar"                  movimiento 2 minutes ago  Up 2 minutes (healthy)             8080/tcp
mexi-banco-v06-notificacion-1      mexi-banco-v06-notificacion        "java -jar app.jar"                  notificacion 2 minutes ago  Up 2 minutes (healthy)             8080/tcp
mexi-banco-v06-spei-1              mexi-banco-v06-spei                "java -jar app.jar"                  spei        2 minutes ago  Up 2 minutes (healthy)             8080/tcp
mexi-banco-v06-transferencia-1     mexi-banco-v06-transferencia       "java -jar app.jar"                  transferencia 2 minutes ago  Up 2 minutes (healthy)             8080/tcp
```

Captura 9: Contenedores corriendo correctamente y demás servicios sin puertos expuestos

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ docker ps --filter name=cuenta --format '{{.ID}} {{.Names}} {{.Ports}}' # las tres copias, su ID y sus puertos
02fdd7faa1a mexi-banco-v06-cuenta-2 8080/tcp
74a7fcb3464 mexi-banco-v06-cuenta-3 8080/tcp
e2c9fa37c675 mexi-banco-v06-cuenta-1 8080/tcp
```

Captura 10: tres contenedores, cada uno con su ID y con 8080/tcp sin flecha

THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

DS Replicas

localhost

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - e2c9fa37c675:cuenta:8080 , 74a7fcb3464:cuenta:8080 , 02fdd7faa1a:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - 006f5d0ff18c:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 1b5fee2cbcd8:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 0e8befce0953:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 7dd0a7c7564e:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 4d6e42563a31:transferencia:8080

Captura 11: servicio CUENTA con estado UP(3)

4.2 Prueba de Balanceo de Carga (Load Balancing)

- **Objetivo:** Comprobar que el API Gateway interactúa correctamente con Eureka para distribuir equitativamente el tráfico entrante entre las diferentes réplicas de un microservicio, evitando la sobrecarga de un solo nodo.
- **Ejecución:**
 - Se monitorearon los registros (*logs*) en tiempo real de las instancias del microservicio mediante el comando `docker compose logs -f cuenta`.
 - Se simuló una ráfaga de tráfico realizando múltiples peticiones de recarga consecutivas al Gateway a través de la ruta `http://localhost:8080/cuentas`.
- **Verificación:** Se observó en la salida de la terminal cómo las peticiones eran atendidas de forma intercalada por los tres contenedores distintos (ej. cuenta-1, cuenta-2 y cuenta-3). Esto demuestra el correcto funcionamiento del algoritmo de balanceo (*Round-Robin*).

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ docker compose logs -f cuenta | grep atendida # Ctrl+C para salir
cuenta-3 | 2026-10-09T20:39:55.432Z INFO 1 --- [cuenta] [nio-8080-exec-9] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 74a7fcb3464
cuenta-3 | 2026-10-09T20:40:04.300Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 74a7fcb3464
cuenta-3 | 2026-10-09T20:40:04.723Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 74a7fcb3464
cuenta-2 | 2026-10-09T20:39:59.556Z INFO 1 --- [cuenta] [nio-8080-exec-9] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 02fdd7faa1a
cuenta-2 | 2026-10-09T20:40:04.445Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 02fdd7faa1a
cuenta-2 | 2026-10-09T20:40:05.032Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 02fdd7faa1a
cuenta-1 | 2026-10-09T20:39:43.674Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por e2c9fa37c675
cuenta-1 | 2026-10-09T20:39:59.380Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por e2c9fa37c675
cuenta-1 | 2026-10-09T20:40:04.573Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por e2c9fa37c675
cuenta-1 | 2026-10-09T20:40:05.268Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por e2c9fa37c675
cuenta-2 | 2026-10-09T20:55:34.176Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 02fdd7faa1a
cuenta-2 | 2026-10-09T20:55:39.673Z INFO 1 --- [cuenta] [nio-8080-exec-9] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 02fdd7faa1a
cuenta-3 | 2026-10-09T20:55:42.831Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 74a7fcb3464
cuenta-1 | 2026-10-09T20:55:49.630Z INFO 1 --- [cuenta] [nio-8080-exec-2] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por e2c9fa37c675
cuenta-2 | 2026-10-09T20:55:51.355Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 02fdd7faa1a
cuenta-3 | 2026-10-09T20:55:56.106Z INFO 1 --- [cuenta] [nio-8080-exec-4] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/5071791566166 atendida por 74a7fcb3464
```

Captura 12: logs en vivo mostrando contenedores intercalados correctamente

- **Pregunta de análisis:** Al tener 3 réplicas de cuentas, ¿el microservicio de

transferencia también reparte la carga cuando llama a cuenta internamente? Revisando el archivo `docker-compose.yml` detectamos que el servicio **transferencia** tenía declarada la variable **CUENTA_URL**: <http://cuenta:8080>. Esto obliga a la aplicación a usar la red estadística de Docker en lugar de consultar el directorio de Eureka, por lo que las peticiones internas por ahora no se están balanceando.

5. Tolerancia a Fallos: ¿Y si una copia se muere? (16:45 hrs)

- **Objetivo:** Evaluar el comportamiento del sistema (API Gateway y Eureka) cuando una instancia de un microservicio falla de manera abrupta (sin enviar una señal de apagado ordenado).
- **Predicción y Análisis:**
 - **¿Van a fallar todas las consultas, ninguna o algunas?** Dado que el objetivo principal de hacer las copias es balancear la carga y precisamente que el servicio siga funcionando así una copia se caiga, pues no debería fallar ninguna. A excepción de las que estaban usando el servicio en ese mismo instante.
 - **¿Durante cuánto tiempo?** Durante aproximadamente 15 a 20 segundos, ya que el directorio depende de los latidos que se mandan de forma periódica, al hacer el kill de la instancia, esta no se apaga de la misma forma que con un stop, muere de forma abrupta, no se despide, por lo que Eureka debe esperar a que se venza el plazo de tolerancia para declararla muerta.
 - **¿Dejan de fallar en cuanto desaparece del directorio, antes, o después?** Dejan de fallar después puesto que en el caché del gateway, donde se encuentran las rutas de Eureka, sigue teniendo las rutas que ya murieron y precisamente dejan de fallar hasta que se sincroniza la lista con Eureka y deja de mandarle tráfico.
- **Ejecución:**
 - En la Terminal 1, se dejó corriendo un ciclo for infinito que realizaba una consulta GET por segundo al API Gateway, imprimiendo la hora y el código de respuesta HTTP.
 - En la Terminal 2, se simuló una caída repentina ejecutando el comando `docker kill mexi-banco-v06-cuenta-1` a las 16:45:17.

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ date +%T
16:45:17

emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ docker kill mexi-banco-v06-cuenta-1
mexi-banco-v06-cuenta-1
```

Captura 13: Terminal 2

```
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
$ for i in {1..60}; do
  echo "$(date +%T) $(curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
  sleep 1
done
16:45:02 200
16:45:04 200
16:45:05 200
16:45:07 200
16:45:08 200
16:45:09 200
16:45:10 200
16:45:11 200
16:45:13 200
16:45:14 200
16:45:15 200
16:45:16 200
16:45:17 200
16:45:18 200
16:45:20 200
16:45:21 200
16:45:22 200
```

Captura 14: Terminal 1

Resultados de la prueba:

- **Hora de apagado (docker kill):** 16:45:17.
- **Hora en que CUENTA pasa de UP (3) a UP (2) en Eureka:** Aproximadamente a las 16:45:35 (poco después de que se cumpliera el plazo de 15 segundos sin recibir *heartbeats*).
- **Comportamiento observado en los curl:**
 - Durante los primeros 14 segundos posteriores al apagado (de 16:45:17 a 16:45:31), las respuestas siguieron marcando un código 200.
 - Exactamente a las **16:45:32** (justo cuando expiró el plazo de Eureka) apareció el primer error 500.
 - A partir de ahí, se observaron errores 500 intermitentes durante un lapso de **28 segundos** (el último error registrado fue a las 16:46:00).
 - A partir de las **16:46:03**, el gateway finalmente actualizó su lista de instancias vivas y las consultas se estabilizaron por completo, volviendo a arrojar únicamente códigos 200.

- **Conclusión de los tiempos:** En primer lugar, el directorio Eureka esperó su plazo de 15 segundos (hasta las 16:45:32) para borrar la instancia. Luego, el Gateway tardó unos 30 segundos extra en actualizarse. Durante toda esa ventana, el Gateway seguía mandando peticiones a una copia muerta, lo que provocaba esos errores intermitentes que duraron unos segundos apareciendo.

6. Cierre (17:15 hrs)

Conclusión general de la práctica:

- El directorio resuelve la problemática de tener que memorizar la dirección IP de cada servicio, dado que estas cambian de manera dinámica constantemente no es viable anotarlas manualmente siempre. El directorio sirve para que los servicios se registren con su nombre y la dirección IP que tienen actualmente, así otros servicios se comunican con el directorio buscando únicamente por medio de su nombre.
- El gateway resuelve la necesidad de los clientes externos de conocer la dirección IP exacta del servicio que desean consultar, además de evitar que accedan a funciones que son internas. El gateway administra las consultas y se comunica con el directorio para poder dirigir las a la ubicación pertinente del servicio.
- El balanceador resuelve el problema de gestionar las solicitudes de manera equitativa a través de todas las copias, evitando que todas se vayan a una sola a la vez y se generen errores. Emplea el algoritmo de balanceo *Round-Robin*.
- Cuando murió una copia vi que cuando dejamos el ciclo for que realizaba una consulta GET por segundo al API y posteriormente matamos la copia, tardaron unos 15 segundos aproximadamente en aparecer los primeros errores 500, que es precisamente el plazo de espera que se da para que se dé por muerta una copia al no recibir más el heartbeat del servicio. Posterior a esto se demoró unos 28 segundos más en estabilizarse en lo que el gateway actualizaba su lista de rutas con el directorio Eureka, una vez que también ya se había retirado.

7. Tarea (18:00)

```
== 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta ==
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009161154301","claveDestino":"002180009161154399","monto":10}'
{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 002180009161154399","instance":"/transferencias","servicio":"cuenta"}
[HTTP 404]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009161154301","claveDestino":"002180009161154302","monto":99999}'
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 002180009161154301","instance":"/transferencias","servicio":"cuenta"}
[HTTP 422]

== 11. Los saldos no se movieron con los errores ==
-> Ana 750.00 (esperado 750) + Beto 700.00 (esperado 700) = 1450.00; SPEI se llevo 50 a otro banco

Fin. Para ver cuanto tarda el directorio en enterarse de una caída: ./demo-v06.sh --latido
emili@BOOK-EV4EELQ1E2 MINGW64 ~/mexi-banco (trabajo-s07)
```